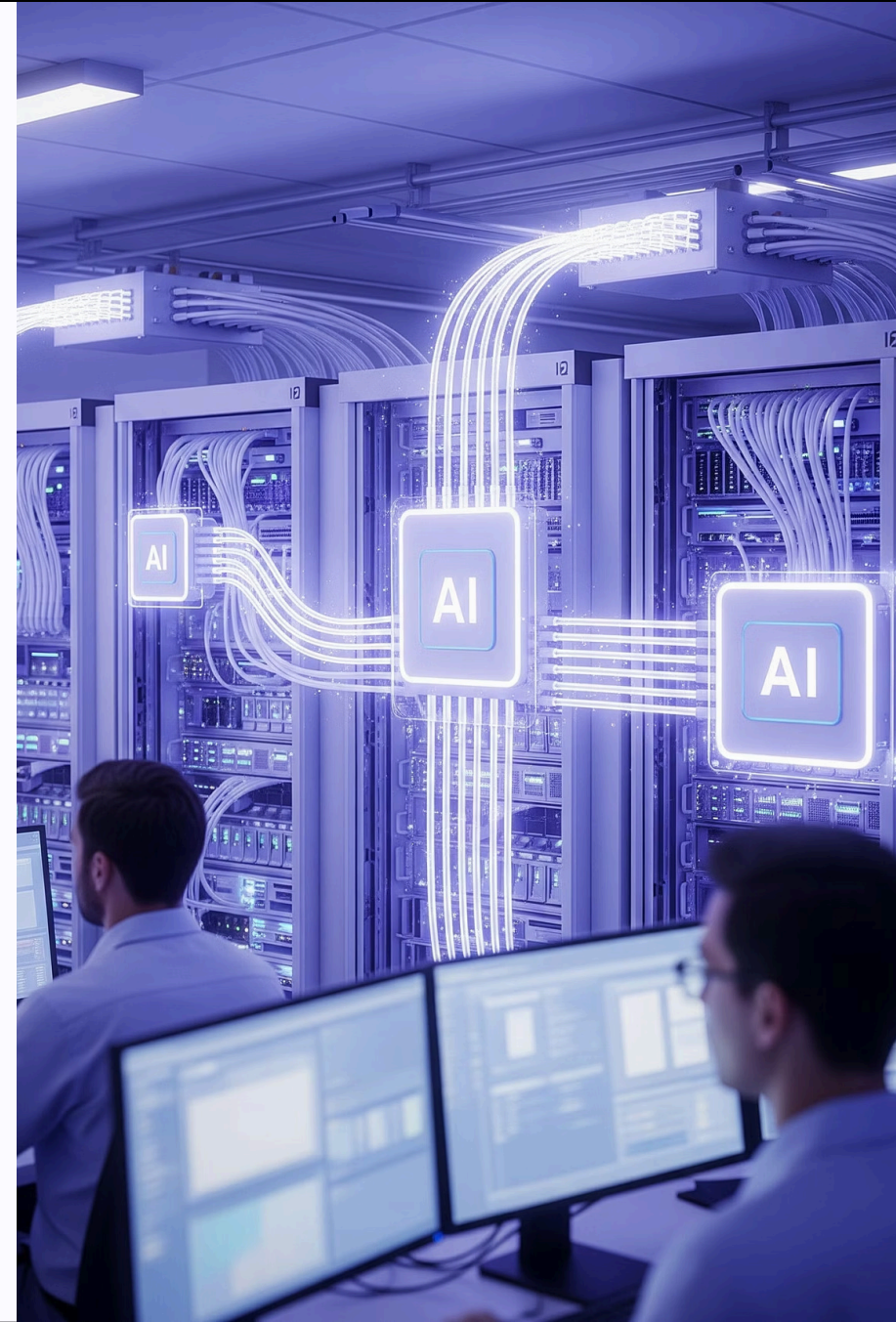


Storage & Data Pipelines for AI Infra

Your model is only as good as your **data flow**. We'll cover object/block/FS storage, schemas, batch & streaming, feature stores, governance, cost, and reliability.



Storage Roles in AI

Object Storage

Datasets, checkpoints, artifacts

Block Storage

Fast training scratch, database volumes

Parallel/Shared FS

Distributed training, POSIX compliance

Metadata Store

Manifests, lineage, registry

Each storage type serves specific roles in your AI infrastructure pipeline, optimized for different performance characteristics and access patterns.

Reference Architecture



The medallion architecture provides clear separation of concerns while enabling comprehensive data governance throughout the pipeline.

Object Storage (Cloud-Native Default)

S3 / GCS / Azure Blob storage provides the foundation of most AI data pipelines:

- Versioning + lifecycle policies (hot → warm → cold)
- Infinite scale with separation of compute and storage
- Cost-effective for datasets of any size

Recommended Layout

```
s3://proj-xyz/  
  bronze/ # raw data  
  silver/ # cleaned data  
  gold/   # features  
  models/ # checkpoints, artifacts  
  logs/   # pipeline logs, audits
```



Block & Shared File Systems

Block Storage

- EBS/PD volumes
- Low-latency training scratch
- Database volumes
- Single-node access

Parallel File Systems

- FSx Lustre, Filestore, BeeGFS
- Multi-node training & checkpointing
- Shared POSIX access
- Higher cost, higher performance

Best practice: Mount for speed during active workloads; archive to object storage for long-term cost efficiency.

K8s Persistent Volumes

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: train-scratch
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 500Gi
  storageClassName: premium-ssd
---
apiVersion: v1
kind: Pod
metadata:
  name: trainer
spec:
  volumes:
    - name: scratch
      persistentVolumeClaim:
        claimName: train-scratch
  containers:
    - name: train
      image: ghcr.io/acme/train:1.0.0
      volumeMounts:
        - name: scratch
          mountPath: /mnt/scratch
```

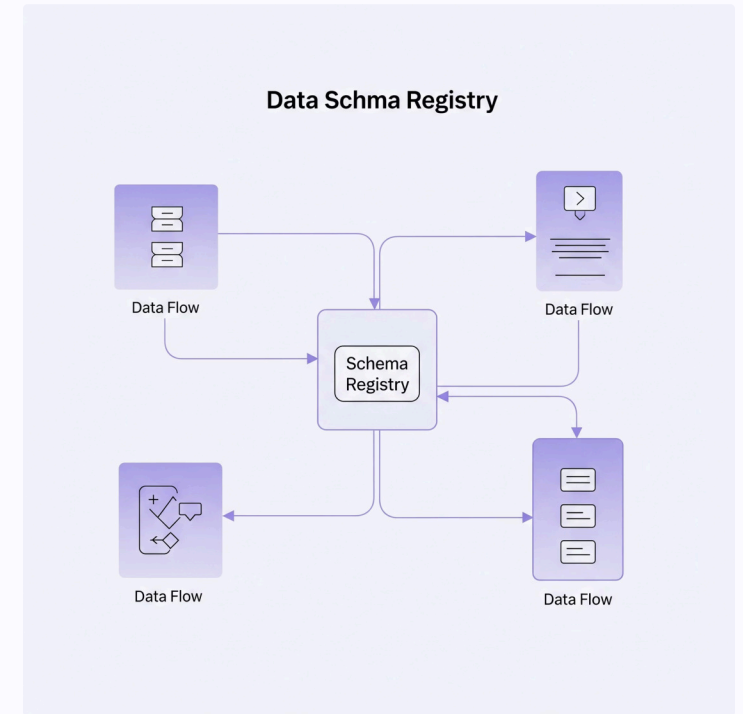
YAML defines persistent storage for Kubernetes workloads, providing reliable storage access to training pods.

Data Modeling & Schemas

Schema-First Development

- Protobuf/Avro/JSON Schema for strict typing
- Schema Registry (e.g., Confluent) for streams
- Immutable, append-only raw data (Bronze)
- Promote data with contracts (Silver/Gold)

Schema evolution should be compatible and versioned, preventing downstream breakage and enabling clean migrations.



Data Quality & Validation



Great Expectations / Deequ

Assert null percentages, value ranges, uniqueness constraints, and relationships



Validation Points

Validate at ingest and during tier promotion (Bronze→Silver→Gold)



Failure Handling

Fail fast, quarantine bad data, alert on anomalies, track data quality metrics

Data quality is not a checkbox but a continuous process. Implement early detection and robust remediation procedures.

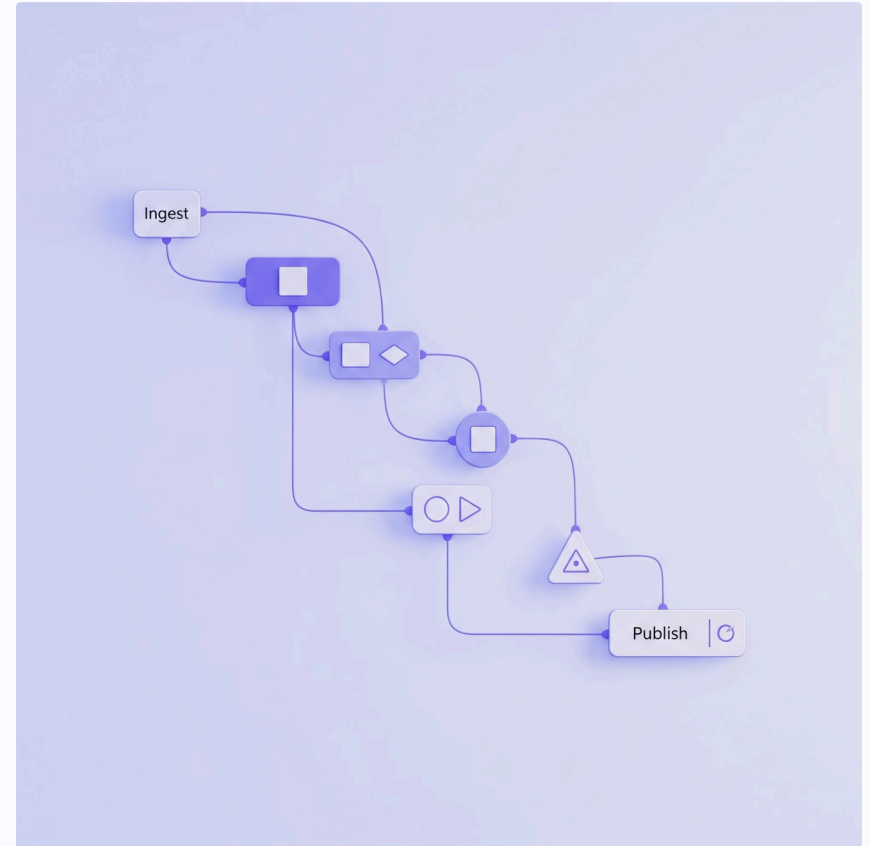
Batch Pipelines (ETL/ELT)

Orchestration tools like **Airflow**, **Dagster**, or **Argo Workflows** coordinate batch data processing.

Typical DAG Flow:

1. Ingest raw data
2. Validate against expectations
3. Clean and normalize
4. Build features
5. Publish to consumers

Store DAG runs + data manifests for complete lineage tracking.



Streaming Pipelines (Real-Time)



Messaging

Kafka / Pub/Sub / Kinesis for event streams



Processing

Flink / Spark Structured Streaming for stateful transforms



Semantics

Use Exactly-Once semantics where needed for correctness

Streaming pipelines merge real-time features into your feature store, enabling low-latency ML inference with fresh data.

Key challenge: Maintaining consistency between batch and streaming pipelines while handling late-arriving data.

Feature Store (Training–Serving Parity)

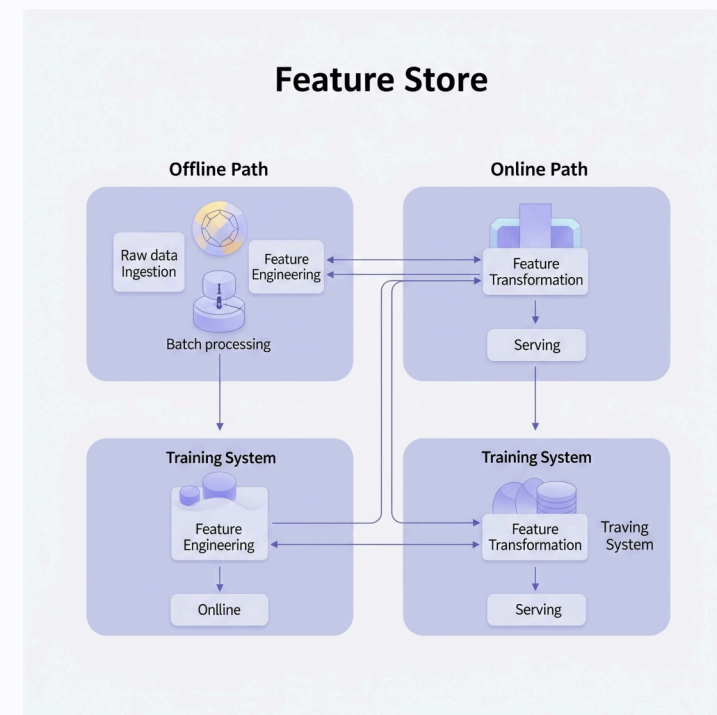
Solutions like **Feast**, **Tecton**, or **Vertex AI Feature Store** solve the critical ML problem of training-serving skew.

Key Benefits:

- Guarantees consistency between offline (train) & online (serve)
- Point-in-time correctness for historical features
- Centralized feature registry and discovery

Example entities: `user_id`, `item_id`

Backends: offline (Parquet/Object), online (Redis/Cassandra)



Feast Quick Sketch (Python)

```
from feast import Entity, FeatureView, Field
from feast.types import Int64, Float32

user = Entity(name="user_id", join_keys=["user_id"])

user_features = FeatureView(
    name="user_stats",
    entities=[user],
    ttl=None,
    schema=[
        Field(name="ctr_30d", dtype=Float32),
        Field(name="views_7d", dtype=Int64)
    ],
    source=... # batch source (Parquet) + optional stream source
)
```

Materialize offline → online for low-latency inference with `feast materialize` or real-time pipelines.

Feature views define what ML features are available, their types, and how to retrieve them.

Example: Airflow DAG

```
@dag(schedule="@hourly")
def build_features():
    raw = ingest_from_kafka()

    # Great Expectations checks
    clean = validate_and_clean(raw)

    silver = dedupe_and_join(clean)

    gold = compute_features(silver)

    publish = materialize_to_feast(gold)

build_features()
```

This DAG:

1. Runs hourly
2. Ingests data from Kafka
3. Validates and cleans
4. Processes through silver tier
5. Computes ML features
6. Publishes to feature store

Governance, Lineage, Catalog

Data Catalog
Amundsen, DataHub, Collibra for
discovery and documentation

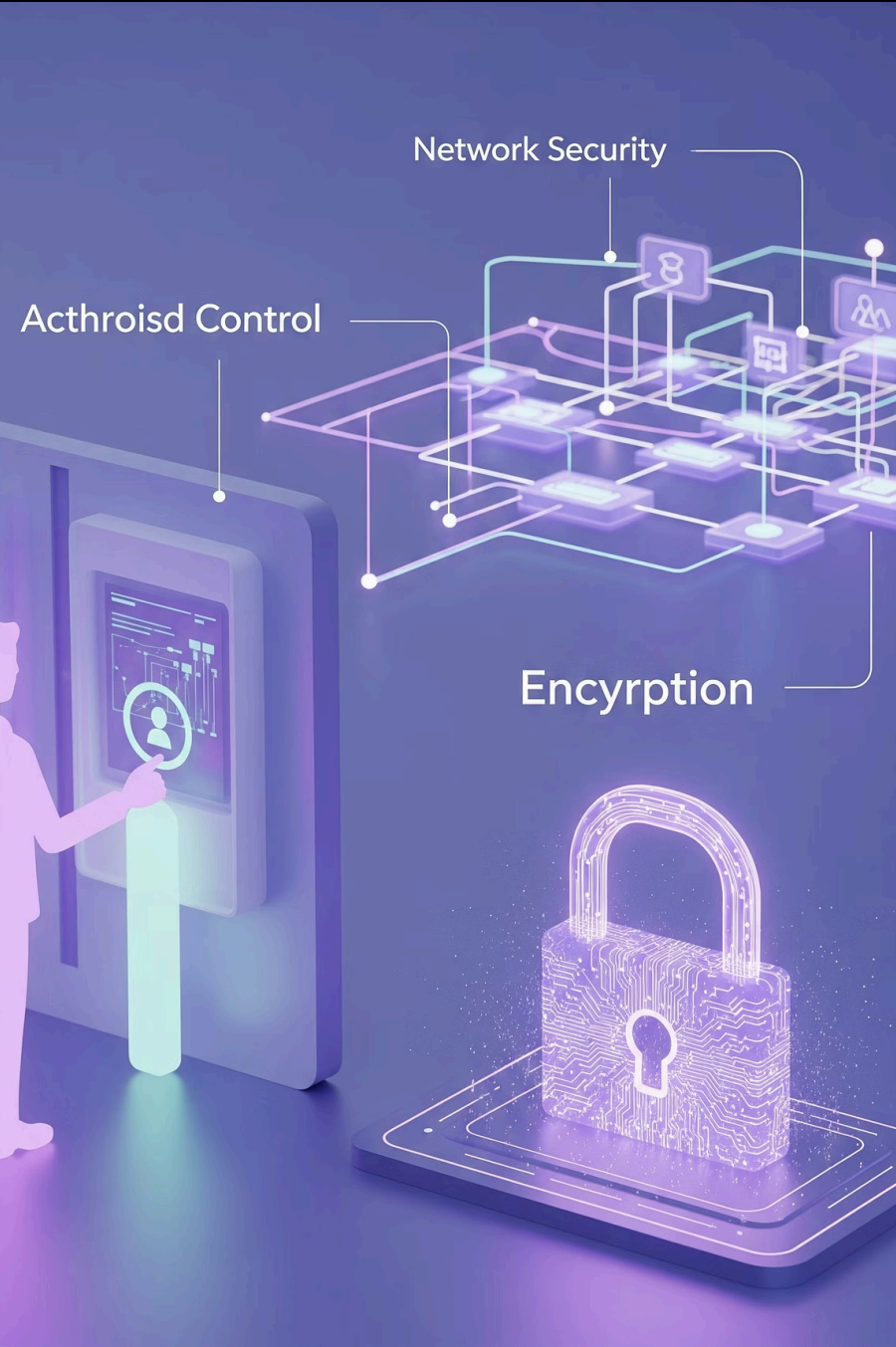
Audit Logs
Compliance tracking for all data
access and modifications



Lineage
OpenLineage, Marquez connects
jobs → datasets → models

Access Control
IAM, table ACLs, row/column-level
security

Governance isn't just about compliance—it enables collaboration through shared understanding of data assets.



Security for Data Layers

Data Protection

- Encrypt in transit (TLS)
- Encrypt at rest (KMS-managed keys)
- Private endpoints/VPC peering → avoid public internet

Access Management

- Secrets in K8s → use external Vault/KMS providers
- PII handling: de-identification, tokenization
- Least-privilege access for all components

Cost Controls (FinOps)

Lifecycle Management

Move Bronze to colder tiers after N days

Efficient Storage

Partition & compress (Parquet + ZSTD/Snappy)

Caching Strategy

Cache hot features online; keep history offline

Metrics Tracking

Monitor \$/TB-month and \$/feature read in dashboards

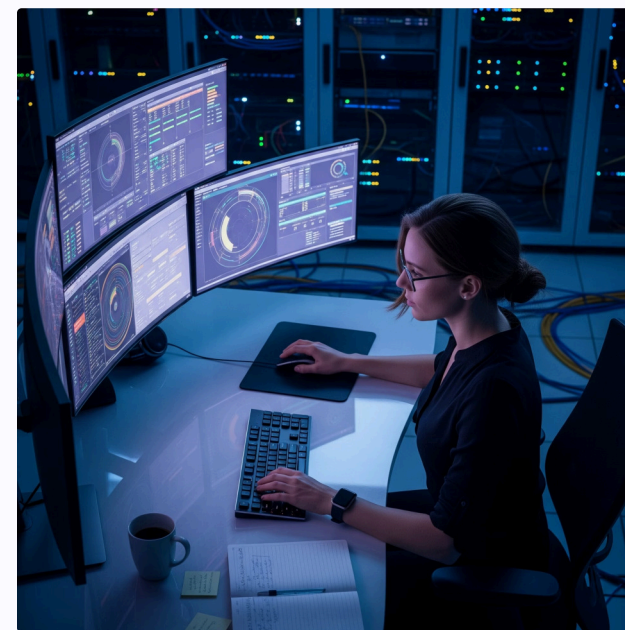
Cloud storage costs can quickly dominate AI infrastructure spend. Implement automated policies to control sprawl and reduce unnecessary replication.

Reliability & Backfills

Pipeline Resilience

- Idempotent pipelines; deterministic outputs by partition/date
- Checkpointing & watermarking for streams
- Backfill strategy: rerun by date range; isolate outputs
- Data contracts to prevent breaking changes

Production ML requires the ability to recover from failures and backfill historical data without disrupting downstream consumers.



Monitoring & SLAs

30m

Latency SLA

Max time from ingest to
feature availability

99.9%

Validation Pass Rate

Percentage of data meeting
quality standards

15m

Freshness

Maximum lag minutes for
critical features

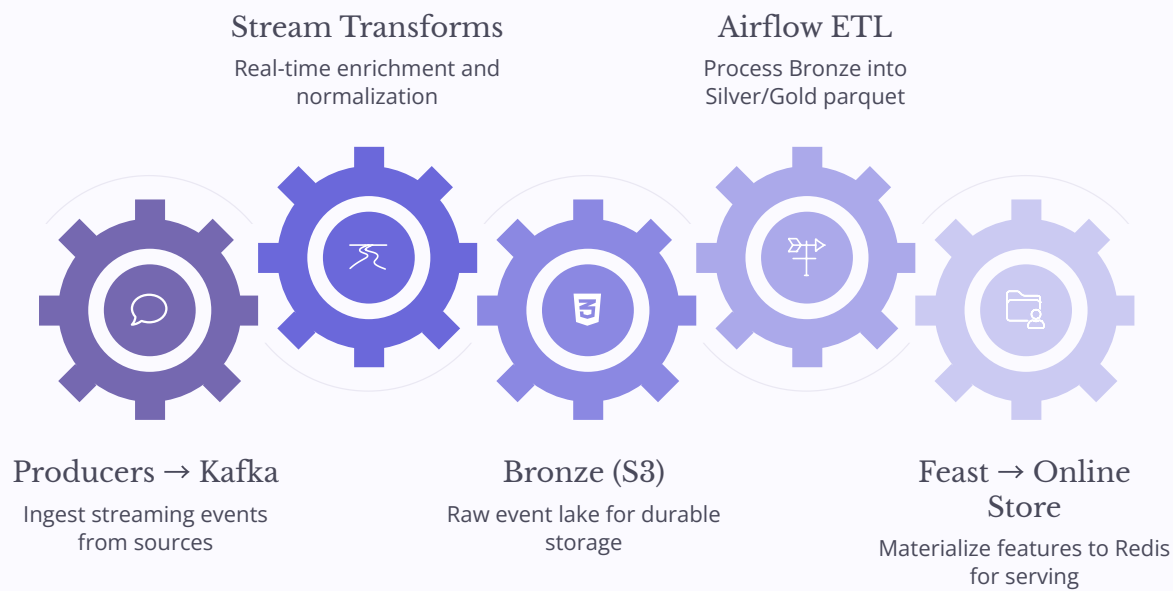
$\pm 5\%$

Drift Threshold

Allowable feature
distribution change

Emit metrics → Prometheus/Grafana; route alerts via PagerDuty based on SLA violations.

Putting It Together



The complete architecture integrates batch and streaming, with clear quality gates and consistent feature access patterns for both training and serving.

Minimal Starter Checklist

1 Storage Foundation

Create buckets & lifecycle rules (bronze/silver/gold)

2 Schema Management

Set up Schema Registry & data validation frameworks

3 Processing Framework

Stand up Airflow (batch) and Kafka (stream)

4 Feature Management

Add Feature Store (Feast) with offline/online stores

5 Operational Readiness

Wire monitoring (freshness, quality) + IAM/KMS security

Start with the minimum viable pipeline, then iterate as you learn what your specific ML use cases require.

Summary

Building reliable AI infrastructure requires:

- Separate **storage roles** and **promotion layers** (Bronze→Silver→Gold)
- Standardize **schemas, validation, lineage**
- Use a **feature store** for train/serve parity
- Bake in **security, cost, reliability** from day one

The quality of your data infrastructure directly impacts the reliability, performance, and maintainability of your ML systems.



Discussion

For your Capstone...

Will you start with **batch-only** pipelines or **hybrid batch+stream**?

Which metrics or product needs drive that choice?

